SQA Workflow — Design to Sign-off

Oct 10, 2026 · @Spriha · Draft v0.1

QA is involved from the first approved design to the final sign-off note, so defects are prevented early rather than found late. This document describes how the team works today within our SDLC, and how each stage runs both without AI and with AI assistance.

Purpose, scope and roles

Scope: every feature from approved Figma design and acceptance criteria (AC) through to QA sign-off. Production release and post-release monitoring are out of scope.

Role	Responsibility in this workflow
Project Manager (PM)	Tracks timelines and progress across all projects and sprints; creates tickets ahead of sprint planning
Product Owner (PO)	Owns the ticket description and acceptance criteria; approves design; answers scope questions
Designer	Owns the Figma design; resolves UI questions raised by QA and Dev
Business Analyst (BA)	Clarifies business rules, edge cases and compliance needs
Developer	Builds the feature; deploys; comments the environment URL and test data; updates ticket status
QA Engineer	Reviews design and AC, writes test cases, executes tests, logs defects, retests, gives sign-off; checks the correct branch is deployed and deploys it through Jenkins when needed
QA Lead	Reviews test coverage and approves the sign-off note

Tools: JIRA (tickets, bugs), Zephyr (test cases, test cycles, coverage), Figma (design), Confluence/Notion (test plans, sign-off notes), Postman (API testing), Jenkins (deployments). UI testing is manual; there is no UI automation.

Sprint cadence

We follow Agile sprints: tickets are picked according to team capacity, and code freezes one week before release so that week is spent on regression testing of the release branch.

Sprint event	When	QA's part
Ticket creation	Before sprint planning	PM creates the tickets in advance so sprint time is not lost to setup
Backlog grooming	Before the ticket enters a sprint	Stages 1 and 2: design and AC review, questions, risk, QA estimate
Sprint planning	Start of sprint	Team picks tickets according to capacity; QA effort is counted in that capacity
Daily stand-up	Every day	Testing progress, blockers, open defects
Development and testing	During the sprint	Stages 3 to 6: test design, handoff, execution, defects and retest
Mid-sprint meeting	Middle of the sprint	Team and PM check whether all tickets can still be completed in the sprint, so the burndown chart stays on track; QA flags testing at risk
Code freeze	One week before release	Release branch is cut; no new features merged, only PO-approved defect fixes
Regression week	The code freeze week	Stage 7: full regression on the release branch
Sprint review and retrospective	End of sprint	QA confirms what is tested and raises process issues

Sprint event	When	QA's part
Release	After regression week	Stage 8 sign-off note required before release

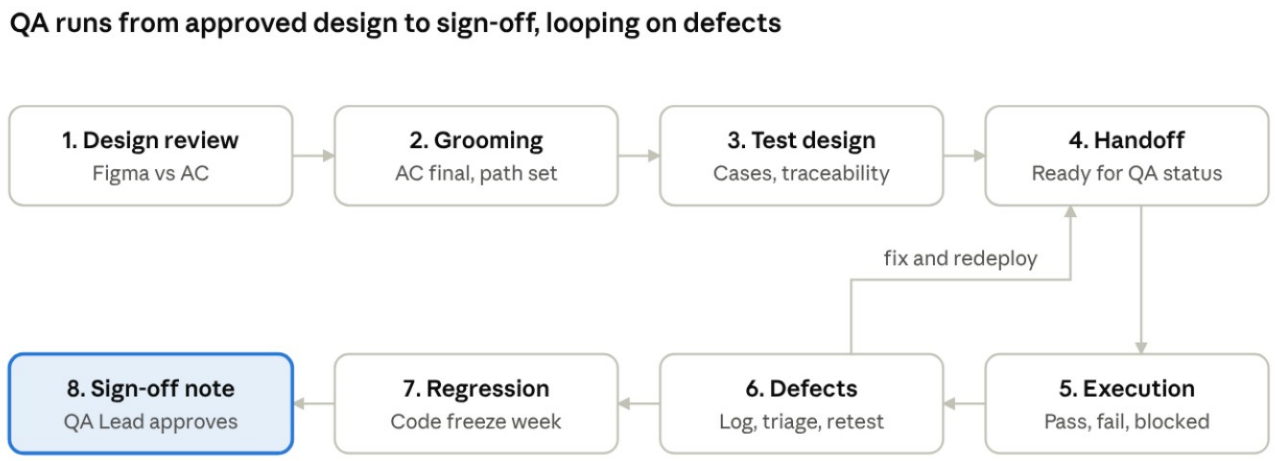
Scope changes during the sprint

A missing AC or new scope found during development or testing goes into a separate new ticket; the original ticket is never changed once work has started.

- Applies to tickets in progress, tickets already passed to QA, and tickets marked Done.
- The new ticket is created (usually by the PM), linked to the original, and tracked on its own through this same workflow, with its own test cases linked to it.
- QA tests the original ticket against its original AC only; the new scope is tested under the new ticket.
- A missing AC found during testing is raised as a new ticket, not logged as a bug on the original.

Activity	Without AI	With AI
Raise the gap	QA or Dev describes the missing AC or scope in the team channel or ticket comment	Same
Create the new ticket	PM writes the ticket and links it to the original	AI drafts the ticket description and proposed AC from the discussion; PM and PO review before saving
Plan it	Team decides whether it fits this sprint's capacity or goes to the backlog	Same

Workflow at a glance



SQA workflow · 8 stages, 1 defect loop

Stages 1 to 3 run before and during development; a failed retest loops back through the handoff until the defect is fixed, and only then does the feature move to regression and sign-off.

Stage 1: Figma design and acceptance criteria review

QA reviews the approved Figma design against the AC before grooming, so gaps are raised while they are still cheap to fix.

Entry: Figma design approved by PO; ticket has a description and AC written by the PO.

Activity	Without AI	With AI
Understand the feature	Read the ticket, AC and every Figma screen manually	AI summarises the ticket and AC into a one-page feature brief; QA verifies it against Figma
Check design vs AC	Walk each screen and match it to each AC line by hand	AI lists AC lines with no matching screen or state, and screens with no AC; QA confirms each item
Find missing states	QA lists empty, error, loading, disabled and permission states from experience	AI suggests missing states and edge cases per screen; QA keeps only the relevant ones

Activity	Without AI	With AI
Check testability	QA flags vague AC wording such as "fast" or "user-friendly"	AI flags ambiguous or untestable AC lines and proposes measurable wording
Record questions	QA writes questions in the ticket or a shared doc	AI drafts the question list; QA edits it before posting

Exit / output: a list of design and AC questions attached to the ticket, ready for grooming.

Stage 2: Backlog grooming

In grooming, QA brings the Stage 1 questions so the team agrees on scope, AC and risk before estimation.

Entry: Stage 1 question list ready; PO, Designer, BA, Dev and QA attend.

Activity	Without AI	With AI
Walk through the feature	PO presents the ticket and Figma; QA raises questions one by one	Same, with the AI feature brief shared in advance so the session starts with questions, not reading
Clarify AC	Team agrees answers; PO updates AC in the ticket	AI drafts revised AC from the meeting notes; PO reviews and updates the ticket
Assess risk	QA and Dev judge impact on existing flows, roles, data and compliance	AI lists modules and flows likely affected; team confirms the real impact
Estimate QA effort	QA estimates from past similar tickets	AI suggests an estimate based on test case count and complexity; QA adjusts
Choose QA path	Team agrees Pilot QA (default) or Dev QA (high-risk changes only)	Same; the decision stays with the team
Capture notes	QA writes meeting notes manually	AI produces meeting notes and action items; QA checks and posts them

Exit / output: AC finalised by the PO, open questions resolved or assigned, QA estimate and QA path agreed, ticket marked ready for sprint.

Stage 3: Test planning and test case design

While development is in progress, QA writes the test cases so testing can start the moment the build is handed over.

Entry: ticket picked in sprint planning according to team capacity, with final AC.

Activity	Without AI	With AI
Test scope	QA lists in-scope and out-of-scope items, roles and environments	AI drafts the scope from the ticket and grooming notes; QA corrects it
Test scenarios	QA derives scenarios from each AC line	AI generates scenarios per AC line; QA removes duplicates and adds business context
Test cases	QA writes test cases in Zephyr with steps, test data and expected results	AI drafts test cases in the Zephyr format; QA reviews every step and expected result
Negative, boundary and validation cases	QA applies boundary value analysis and equivalence partitioning by hand	AI proposes boundary, invalid input and role-permission cases; QA confirms against business rules
API tests	QA builds Postman requests and collections from the API spec	AI drafts Postman requests and assertions from the spec; QA runs and fixes them
Traceability	QA links each Zephyr test case to the JIRA story and maps it to an AC line	AI builds the AC-to-test-case matrix and flags uncovered AC lines
Peer review	QA Lead or a peer reviews the test cases	Same; AI can do a first-pass review for gaps before the human review

Exit / output: reviewed test cases in Zephyr, linked to the JIRA ticket, test data identified, AC-to-test-case traceability complete.

Stage 4: Dev-to-QA handoff

A build reaches QA only through the handoff SOP: the developer deploys, comments the environment details, and moves the ticket to the agreed QA status. QA then checks in Jenkins that the correct branch is deployed, and deploys it themselves if it is not.

Entry: development complete and deployed.

Activity	Without AI	With AI
Deploy	Developer deploys to Pilot (default) or Dev (high-risk only)	Same
Handoff comment	Developer adds the environment URL and test data in a ticket comment	Same; AI can check the comment and flag a missing URL or test data
Status update	Developer moves the ticket to "Ready for Pilot QA" or "Ready for Dev QA"	Same
Verify deployed branch	QA checks in Jenkins that the correct branch is deployed to the environment	Same
Deploy if needed	If the correct branch is not deployed, QA deploys it through Jenkins	Same
Smoke check	QA opens the build and checks the feature loads and the main flow works	Same; a Postman smoke collection can check the main APIs first
Reject incomplete handoff	QA returns the ticket if the URL, test data or build is missing or broken	Same

Exit / output: build accepted by QA, ticket in a Ready for QA status.

Stage 5: Test execution

QA runs every planned test case on the handed-over build and records a pass, fail or blocked result with evidence.

Entry: build accepted in Stage 4; reviewed test cases ready.

Activity	Without AI	With AI
Create test cycle	QA creates a Zephyr test cycle for the ticket and adds its test cases	Same
Functional UI testing	QA executes each test case in the cycle manually across all user roles	Same; AI is not a substitute for hands-on execution
API testing	QA runs Postman collections and checks responses, status codes and data	Same; AI helps read failed responses and suggests likely causes
Design check	QA compares screens to Figma by eye	AI-assisted visual comparison highlights differences; QA judges which matter
Exploratory testing	QA explores beyond the test cases using experience	AI suggests exploratory charters and risky areas; QA explores
Audit trail and data checks	QA verifies audit trails, data integrity and transaction accuracy	Same; never delegated to AI
Record results	QA marks each test case Pass, Fail or Blocked in the cycle and attaches evidence	Same; QA records every result
Track coverage	QA reads executed vs total test cases from the cycle	AI turns the cycle's numbers into a daily status update; QA verifies them

Exit / output: every test case in the Zephyr cycle has a result; coverage is read from the cycle; failures are logged as JIRA bugs and linked to the failed test cases.

Stage 6: Defect management and retest

Every failure becomes a JIRA defect with clear steps and evidence, and loops back to the developer until it passes retest.

Entry: at least one failed test case from Stage 5.

Activity	Without AI	With AI
Log defect	QA logs a JIRA bug with title, steps, expected vs actual, environment, role and evidence, and links it to the failed Zephyr test case	AI drafts the defect from QA's notes and screenshots; QA checks every field and links the bug to the failed test case
Severity and priority	QA sets severity; PO confirms priority	AI suggests severity from impact; QA and PO decide
Duplicate check	QA searches JIRA for similar defects	AI finds likely duplicates; QA confirms
Triage	QA, Dev and PO agree fix, defer or reject	Same
Fix and redeploy	Developer fixes, redeploys and comments per the handoff SOP	Same
Retest	QA re-executes the linked test case in the Zephyr cycle, then closes or reopens the bug	Same
Impact retest	QA retests related test cases around the fix	AI suggests which related cases are most likely affected; QA decides

Exit / output: all critical and high defects closed; any remaining defects deferred with PO approval.

Stage 7: Code freeze and regression week

Code freezes one week before release, and QA spends that week running regression on the release branch so no sprint change breaks existing flows.

Entry: Stage 6 exit met for every ticket in the release; release branch cut, and QA has confirmed in Jenkins that the release branch is deployed (deploying it if not).

Activity	Without AI	With AI
Code freeze	Release branch cut one week before release; after that only PO-approved defect fixes are merged	Same
Select regression scope	QA picks regression cases covering every ticket in the release plus core flows	AI ranks regression cases by the areas the release's tickets touch; QA finalises the list
Run regression on release branch	QA creates a regression test cycle in Zephyr and runs manual regression and the Postman API collections on the release branch build	Same; AI helps read failed API responses
Freeze-week defects	QA logs and triages; a fix is merged to the release branch only if approved, then retested	AI drafts defect reports; QA checks every field
UAT support	QA walks the PO through the release against the AC	AI prepares a UAT checklist from the AC; QA walks the PO through it
Final retest	QA reruns cases affected by fixes merged during the freeze	AI suggests which cases those fixes affect; QA decides

Exit / output: regression passed on the release branch, PO has accepted the release against the AC.

Stage 8: QA sign-off note

QA issues a written sign-off note once all exit criteria are met; the QA Lead approves it and the PO receives it as the record that the feature is ready for release.

Sign-off criteria:

- [ ] All planned test cases executed
- [ ] No open critical or high defects

- [ ] Remaining defects deferred with PO approval and listed in the note
- [ ] Regression passed
- [ ] All Zephyr test cases linked to the JIRA ticket, and every AC line traced to at least one passed test case
- [ ] PO acceptance received

Sign-off note contents:

Section	What it contains
Feature	Ticket ID, feature name, sprint, release
Environment	Pilot or Dev, build version, test dates
Scope tested	Modules, user roles, browsers or devices covered
Out of scope	What was not tested and why
Results	Test cases total, passed, failed, blocked; Postman API test results
Defects	Total raised, closed, deferred, with IDs of deferred ones
Known issues and risks	Anything the release team or PO should know
Recommendation	Go, or Go with known issues
Approval	QA Engineer, QA Lead, PO names and dates

Activity	Without AI	With AI
Compile results	QA takes execution counts and coverage from the Zephyr test cycles and bug counts from JIRA	AI compiles counts and defect lists into the template; QA verifies each number against Zephyr and JIRA
Write the note	QA writes the summary, risks and recommendation	AI drafts the summary and risk section; QA rewrites as needed and owns the recommendation
Approve and share	QA Lead approves; note posted in Confluence and linked on the ticket	Same

With vs without AI: summary and guardrails

The stages, owners and sign-off criteria are the same in both processes; AI speeds up drafting and analysis, while QA keeps every decision and every test verdict.

Aspect	Without AI	With AI
Who decides	QA, PO, Dev	QA, PO, Dev (unchanged)
Where time goes	Writing test cases, defects, notes and reports	Reviewing and correcting AI drafts; more time for exploratory and risk-based testing
Coverage gaps	Depend on individual experience	AI suggests extra edge cases; QA filters them
Main risk	Missed cases under time pressure	Trusting an AI draft without checking it

Guardrails when using AI:

1. Never paste real customer data, account numbers, credentials or production data into an AI tool; use masked or synthetic test data only.
2. Use only company-approved AI tools.
3. QA reviews every AI-generated test case, defect, script and report before it is saved or shared.
4. AI never marks a test as passed; only a human execution result counts.
5. Audit trail, compliance and transaction-integrity checks are always done by QA directly.
6. Every number in the sign-off note is checked against Zephyr and JIRA before approval.